



Progress on Defect Predictor

By: Huy Nguyen and Edward Feng



Implementation

- Exclude all test files and irrelevant files
- Document code extensively
- Update README.md
- Implement `calculate_minor()` but haven't tested

Revisit Factors in the Formula

- Recency (Date committed): 25% -> 22.5%
- Frequency (How often the files are changed): 35% -> 30%
- Number of lines modified 20% -> 15%
 - Lines added + lines deleted
- How many lines of code (loc) in the file 10% -> 15%
- Cyclomatic Complexity 10% -> 17.5%
- Number of minor contributors

```
risk_score = (.225 * r) + ( .3 * f ) + ( .15 * m ) + ( .175 * c ) + ( .15 * loc )
```

Number of Minor Contributors

- Get's the number of contributors with less than 5% for each file in the repo
- Different from other calculation functions

Contributors Count

This metric measures the number of developers that contributed to a file.

The class `ContributorsCount` has two methods:

- `count()` to count the number of contributors who modified a file;
- `count_minor()` to count the number of *minor* contributors who modified a file, i.e., those that contributed less than 5% to the file.

For example:

```
from pydriller.metrics.process.contributors_count import ContributorsCount
metric = ContributorsCount(path_to_repo='path/to/the/repo',
                           from_commit='from commit hash',
                           to_commit='to commit hash')

count = metric.count()
minor = metric.count_minor()
print('Number of contributors per file: {}'.format(count))
print('Number of "minor" contributors per file: {}'.format(minor))
```

will print the number of developers that contributed to each of the modified file in the evolution period `[from_commit, to_commit]` and the number of developers that contributed less than 5% to each of the modified file in the evolution period `[from_commit, to_commit]`.

Optimization?

- Ground truth needs to have more data
 - Closer to actual accuract (Right now accuracy is only 0%, 20%, 50%, 100%)
- Parametrize weights (instead of hardcode)
- Loop through different config.jsons
- Call Pythonminer for each config.json
- Calculate risk scores -> accuracies for x sets of weights
- Store accuracies for each sets of weights across all config.jsons
- Calculate accuracy average
- Return the set of weights with the highest average accuracy



What's next?

- Entering Sprint 6
 - Optimization (if applicable)
 - Test updated ground truth
 - Test `calculate_minors()` and redistribute weight
 - Wrap up documentation for code and README.md
 - Start one page report